

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: *Архитектура компьютера*

Студент: Кабанов Иван Олегович

Группа: НКАбд-03-25

МОСКВА

2025 г.

Оглавление

Цель работы _____	3
7.3. Выполнение лабораторной работы _____	3
7.3.1. Реализация переходов в NASM _____	3
7.3.2. Изучение структуры файла листинга _____	8
7.4. Задание для самостоятельной работы _____	11
1. _____	11
2. _____	14
Вывод _____	18

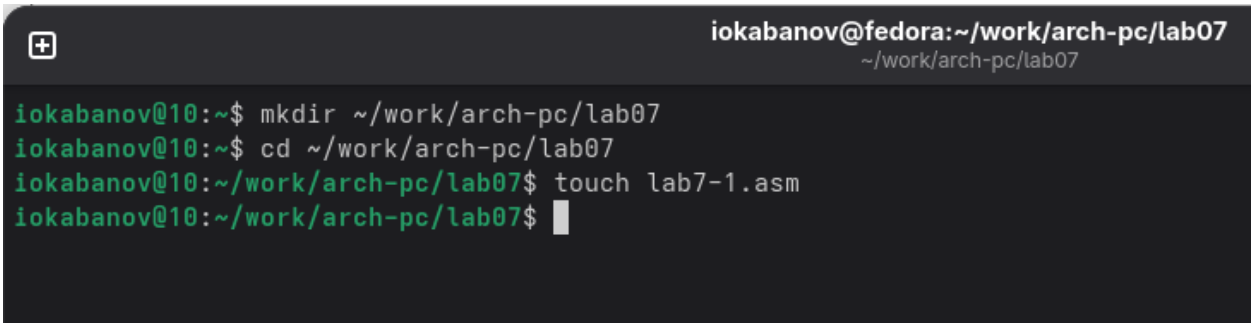
Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

7.3. Выполнение лабораторной работы

7.3.1. Реализация переходов в NASM

Создаем каталог для программ лабораторной работы № 7, переходим в него и создаем файл lab7-1.asm

A screenshot of a terminal window with a dark background. The window title bar shows a plus icon on the left and the text 'iokabanov@fedora:~/work/arch-pc/lab07' on the right. Below the title bar, the terminal displays four lines of commands and their outputs in a green monospaced font. The first line is 'mkdir ~/work/arch-pc/lab07', the second is 'cd ~/work/arch-pc/lab07', the third is 'touch lab7-1.asm', and the fourth is a prompt 'iokabanov@10:~/work/arch-pc/lab07\$' followed by a cursor. The terminal window has a thin grey border.

```
iokabanov@10:~$ mkdir ~/work/arch-pc/lab07
iokabanov@10:~$ cd ~/work/arch-pc/lab07
iokabanov@10:~/work/arch-pc/lab07$ touch lab7-1.asm
iokabanov@10:~/work/arch-pc/lab07$
```

Рис. 1. Создание lab7-1.asm

Вводим в файл lab7-1.asm текст программы из листинга 7.1. Создаем исполняемый файл и запускаем его. Убеждаемся, что результат работы программы верный.

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.NRHBd8
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07
lab7-1.asm [-M--] 47 L:[ 1+26 27/ 27] *(752 / 752b) <EOF>
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:

    jmp _label2

_label1:
    mov eax, msg1 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 1'

_label2:
    mov eax, msg2 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 2'

_label3:
    mov eax, msg3 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 3'

_end:
    call quit ; вызов подпрограммы завершения
```

Рис. 2. Программа из листинга 7-1

```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07
iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
iokabanov@10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
iokabanov@10:~/work/arch-pc/lab07$
```

Рис. 3. Запуск программы из листинга 7-1

Изменяем текст программы в соответствии с листингом 7.2. Создаем исполняемый файл и запускаем его. Убеждаемся, что результат работы программы верный.

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.VKUtAB
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07

lab7-1.asm [-M--] 47 L:[ 1+28 29/ 29] *(785 / 785b) <EOF>
%include 'in_out.asm' ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:

jmp _label2

_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end

_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1

_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'

_end:
call quit ; вызов подпрограммы завершения
```

Рис. 4. Программа из листинга 7-2

```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07

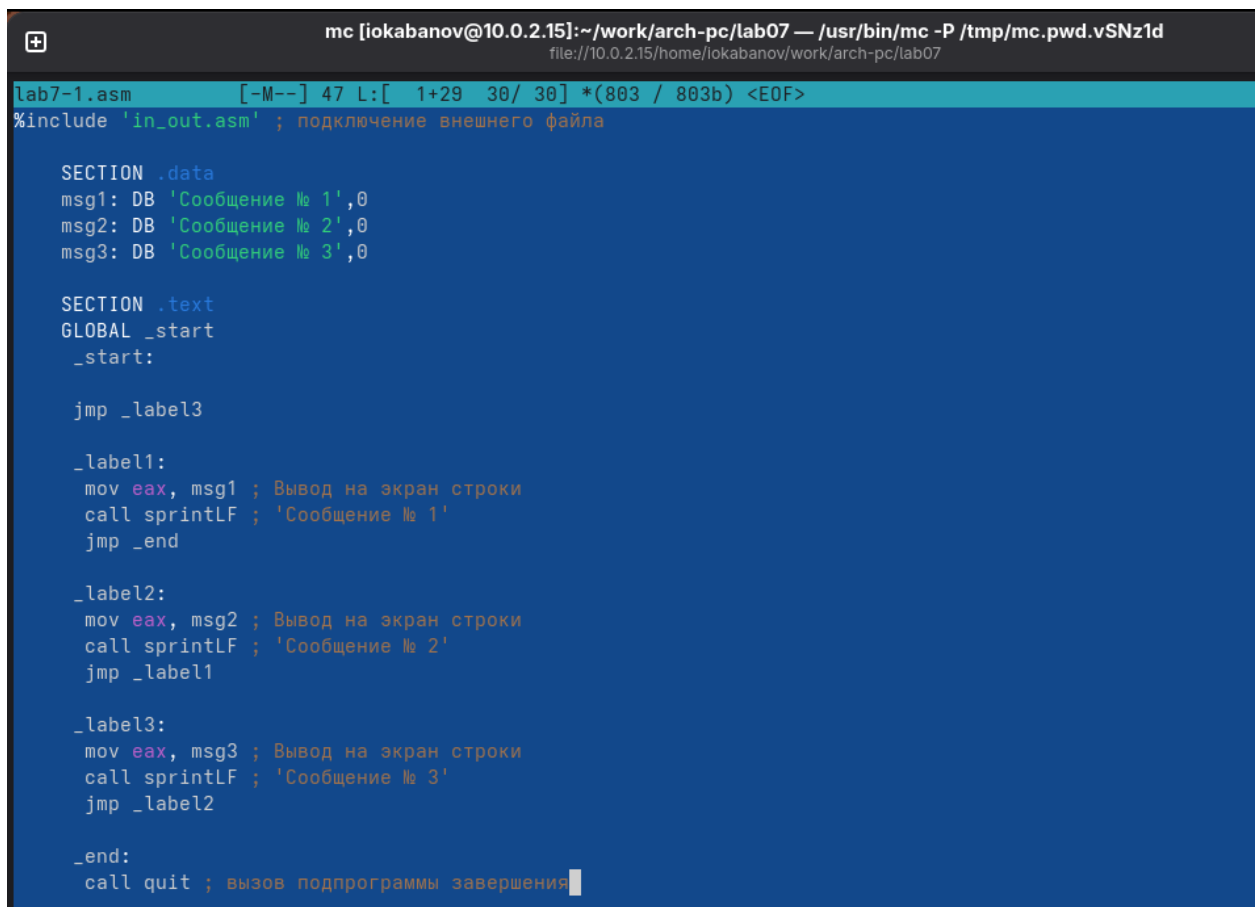
iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm -o lab7-1new.o
iokabanov@10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1new lab7-1new.o
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-1new
Сообщение № 2
Сообщение № 1
iokabanov@10:~/work/arch-pc/lab07$
```

Рис. 5. Запуск программы из листинга 7-2

Изменяем текст программы, меняя инструкцию `jmp _label2` после `_start` на `jmpz _label3` и добавляя инструкцию `jmp _label2` после вывода сообщения №3, чтобы вывод программы был:

Сообщение № 3
Сообщение № 2
Сообщение № 1

Создаем исполняемый файл и запускаем его. Убеждаемся, что результат работы программы верный.



```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.vSNz1d
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07

lab7-1.asm [-M--] 47 L:[ 1+29 30/ 30] *(803 / 803b) <EOF>
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:

    jmp _label3

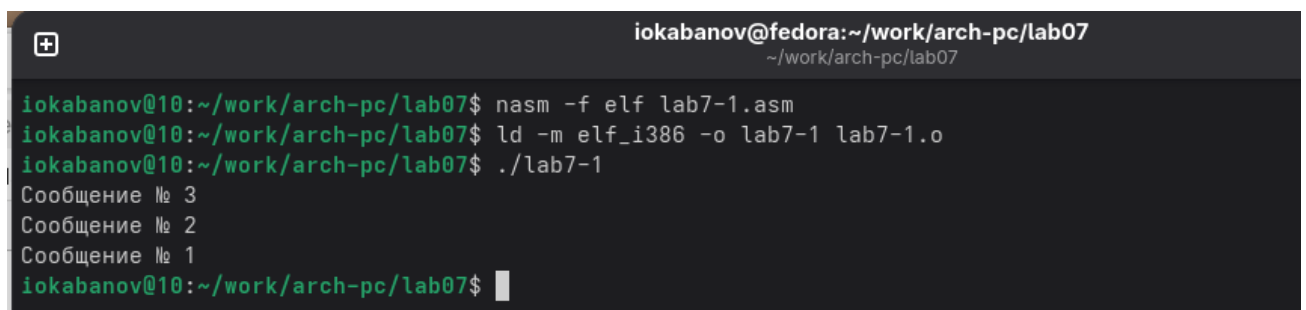
_label1:
    mov eax, msg1 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 1'
    jmp _end

_label2:
    mov eax, msg2 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 2'
    jmp _label1

_label3:
    mov eax, msg3 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 3'
    jmp _label2

_end:
    call quit ; вызов подпрограммы завершения
```

Рис. 6. Измененная программа из листинга 7-2

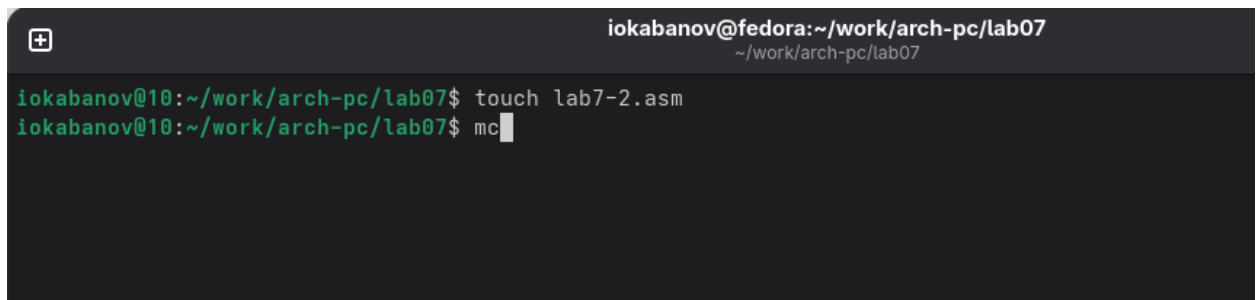


```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07

iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
iokabanov@10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
iokabanov@10:~/work/arch-pc/lab07$
```

Рис. 7. Запуск измененной программы из листинга 7-2

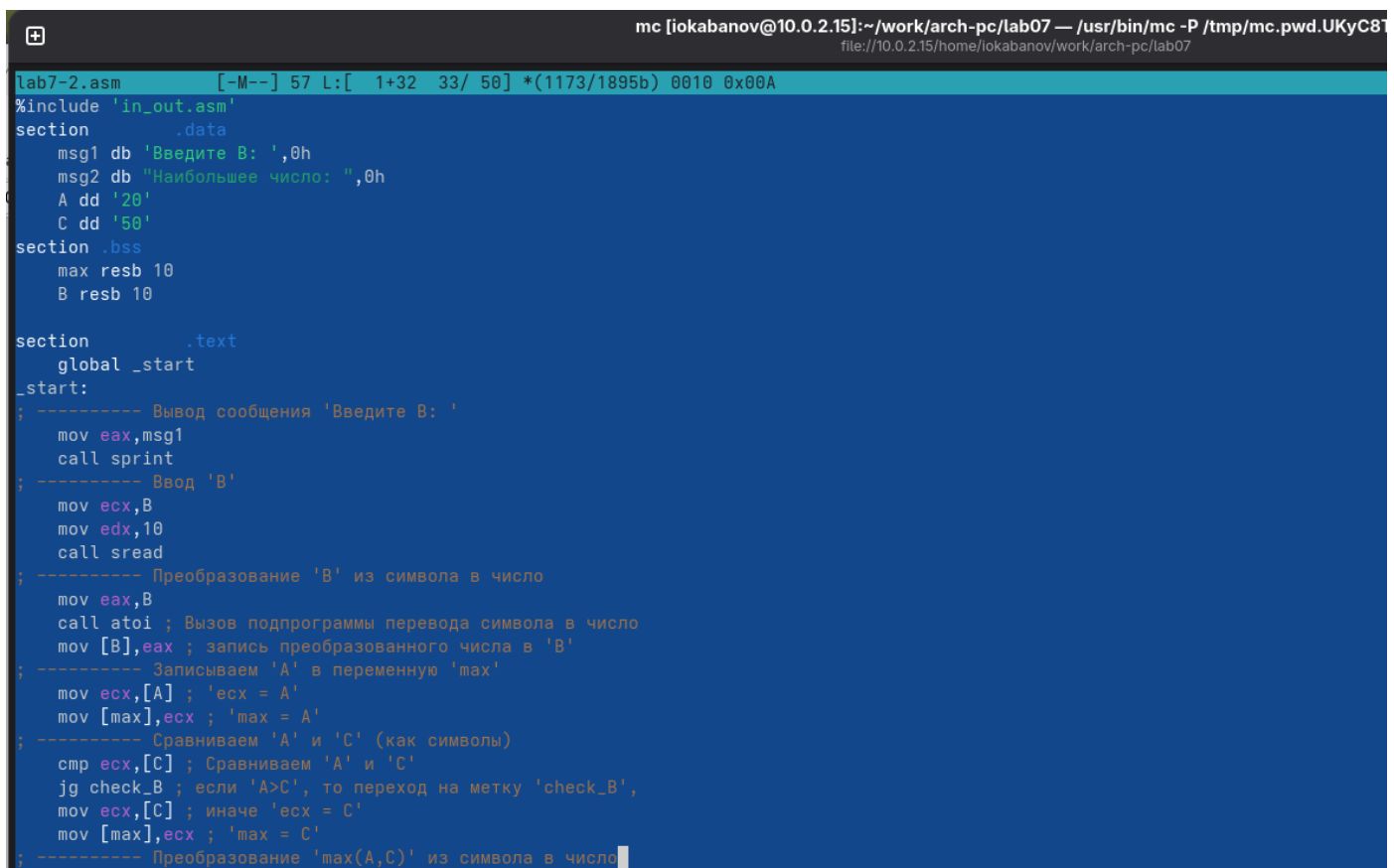
Создаем файл lab7-2.asm в каталоге ~/work/arch-pc/lab07 и вводим в него текст программы из листинга 7-3. Создаем исполняемый файл и проверяем его работу для разных значений В.



```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07

iokabanov@10:~/work/arch-pc/lab07$ touch lab7-2.asm
iokabanov@10:~/work/arch-pc/lab07$ mc
```

Рис. 8. Создаем файл lab7-2.asm



```
lab7-2.asm  [-M--] 57 L:[ 1+32 33/ 50] *(1173/1895b) 0010 0x00A
%include 'in_out.asm'
section      .data
    msg1 db 'Введите В: ',0h
    msg2 db "Наибольшее число: ",0h
    A dd '20'
    C dd '50'
section .bss
    max resb 10
    B resb 10

section      .text
    global _start
_start:
; ----- Вывод сообщения 'Введите В: '
    mov eax,msg1
    call sprint
; ----- Ввод 'В'
    mov ecx,B
    mov edx,10
    call sread
; ----- Преобразование 'В' из символа в число
    mov eax,B
    call atoi ; Вызов подпрограммы перевода символа в число
    mov [B],eax ; запись преобразованного числа в 'В'
; ----- Записываем 'А' в переменную 'max'
    mov ecx,[A] ; 'есх = А'
    mov [max],ecx ; 'max = А'
; ----- Сравниваем 'А' и 'С' (как символы)
    cmp ecx,[C] ; Сравниваем 'А' и 'С'
    jg check_B ; если 'А>С', то переход на метку 'check_B',
    mov ecx,[C] ; иначе 'есх = С'
    mov [max],ecx ; 'max = С'
; ----- Преобразование 'max(А,С)' из символа в число
```

Рис. 9. Программа из листинга 7-3

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.Bwxrye
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07

lab7-2.asm [----] 21 L: [ 9+41 50/ 50] *(1895/1895b) <EOF>
B resb 10

section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx,[B] ; иначе 'ecx = B'
mov [max],ecx
; ----- Вывод результата
fin:
mov eax,msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax,[max]
call iprintlnf ; Вывод 'max(A,B,C)'
call quit ; Выход
```

Рис. 10. Продолжение рис. 9

```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07

iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
iokabanov@10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 79
Наибольшее число: 79
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 8
Наибольшее число: 50
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-2
Введите B: -11
Наибольшее число: 50
iokabanov@10:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 25
Наибольшее число: 50
iokabanov@10:~/work/arch-pc/lab07$
```

Рис. 11. Запуск программы из листинга 7-3

7.3.2. Изучение структуры файла листинга

Создаем файл листинга для программы из файла lab7-2.asm

```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07

iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
iokabanov@10:~/work/arch-pc/lab07$ mcedit lab7-2.lst
```

Рис. 12. Создание файла листинга lab7-2.lst

Открываем файл листинга lab7-2.lst с помощью mcedit.

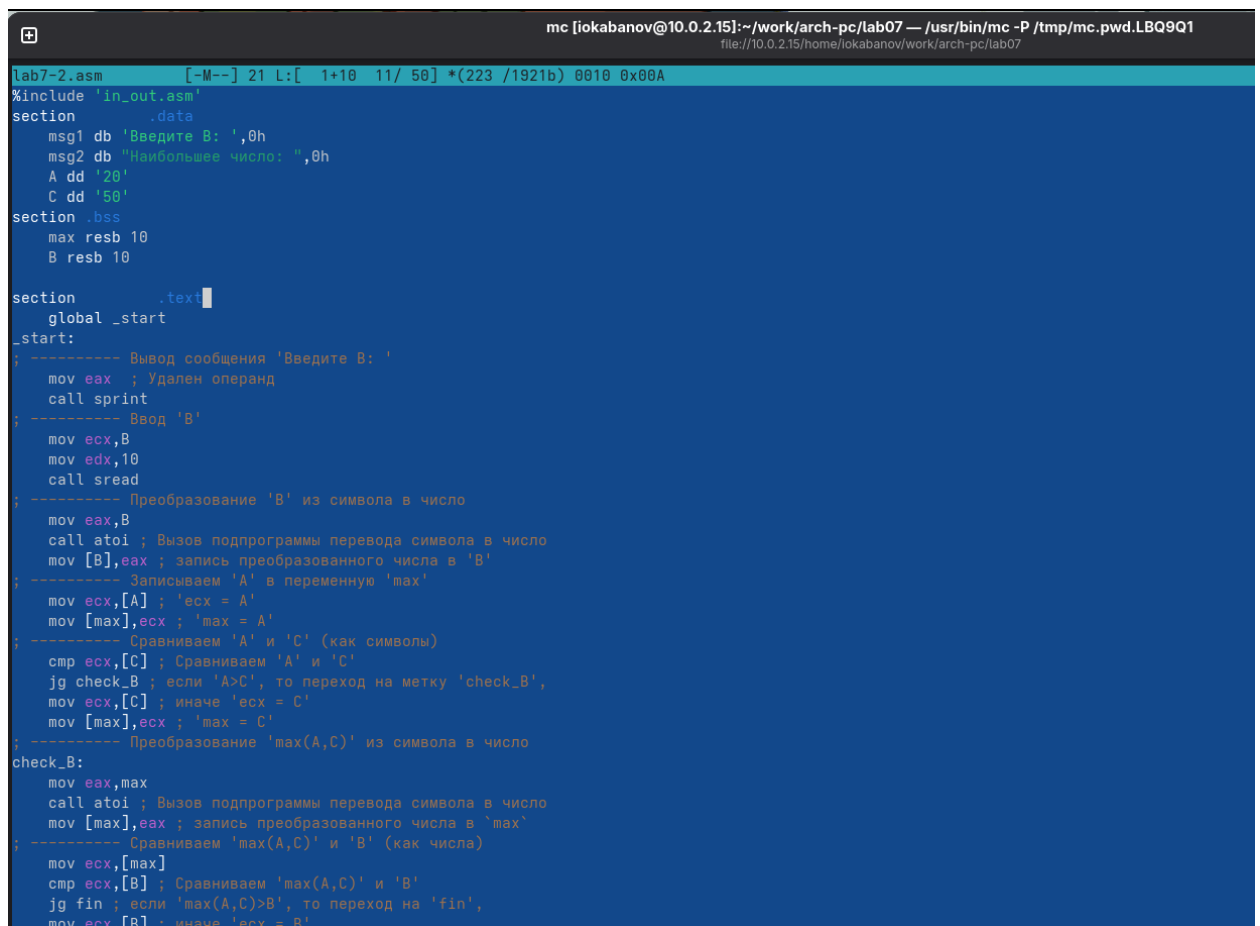
Объяснение строк 14-16:

- Строка 14 (Адрес 0000000B): По этому смещению записана команда `sub eax, ebx`. Она вычитает из текущего адреса (конец строки, `eax`) начальный адрес (начало строки, `ebx`), тем самым вычисляя длину строки и сохраняя результат в `eax`.
- Строка 15 (Адрес 0000000D): По этому смещению записана команда `pop ebx`. Она восстанавливает из стека прежнее значение регистра `ebx`, которое было сохранено там в начале функции, чтобы не нарушить работу основной программы.
- Строка 16 (Адрес 0000000E): По этому смещению записана команда `ret`. Она выполняет возврат из подпрограммы, передавая управление обратно в то место кода, откуда была вызвана эта функция.

```
lab7-2.lst [----] 0 L: [ 1+ 0 1/226] *(0 /14650b) 0032 0x020
1 %include 'in_out.asm'
2 <1> ;----- slen -----
3 <1> ; Функция вычисления длины сообщения
4 <1> slen:
5 00000000 53 <1> push ebx
6 00000001 89C3 <1> mov ebx, eax
7 <1>
8 <1> nextchar:
9 00000003 803800 <1> cmp byte [eax], 0
10 00000006 7403 <1> jz finished
11 00000008 40 <1> inc eax
12 00000009 EBF8 <1> jmp nextchar
13 <1>
14 0000000B 29D8 <1> sub eax, ebx
15 0000000D 5B <1> pop ebx
16 0000000E C3 <1> ret
17 <1>
18 <1>
19 <1> ;----- sprint -----
20 <1> ; Функция печати сообщения
21 <1> ; входные данные: mov eax, <message>
22 <1> sprint:
23 0000000F 52 <1> push edx
24 00000010 51 <1> push ecx
25 00000011 53 <1> push ebx
26 00000012 50 <1> push eax
27 00000013 E8E8FFFFFF <1> call slen
28 <1>
29 00000018 89C2 <1> mov edx, eax
30 0000001A 58 <1> pop eax
31 <1>
32 0000001B 89C1 <1> mov ecx, eax
33 0000001D B801000000 <1> mov ebx, 1
34 00000022 B804000000 <1> mov eax, 4
35 00000027 CD80 <1> int 80h
36 <1>
37 00000029 5B <1> pop ebx
38 0000002A 59 <1> pop ecx
39 0000002B 5A <1> pop edx
40 0000002C C3 <1> ret
41 <1>
```

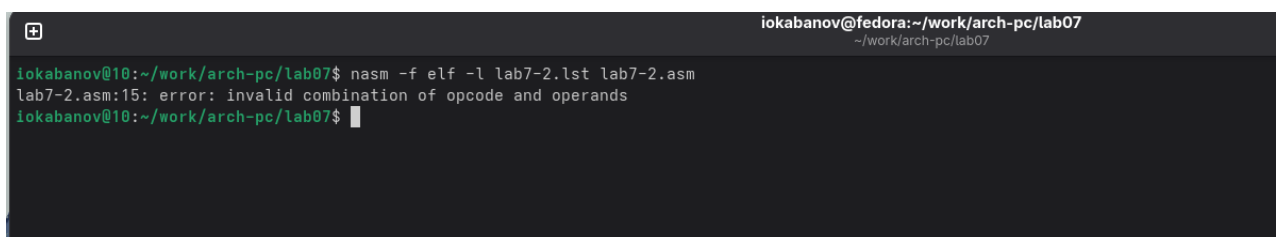
Рис. 13. Фрагмент файла листинга lab7-2.lst

Откроем файл с программой lab7-2.asm и в инструкции mov(находящейся после _start) удаляем операнд msg1. Выполняем трансляцию. В этом случае выдается ошибка invalid combination of opcode and operands, однако файл листинга создается. В файле листинга можно заметить добавление строки с сообщением об ошибке.



```
lab7-2.asm [-M--] 21 L: [ 1+18 11/ 50] *(223 /1921b) 0010 0x00A
#include 'in_out.asm'
section .data
    msg1 db 'Введите B: ',0h
    msg2 db 'Наибольшее число: ",0h
    A dd '20'
    C dd '50'
section .bss
    max resb 10
    B resb 10
section .text
    global _start
_start:
; ----- Вывод сообщения 'Введите B: '
    mov eax ; Удален операнд
    call sprint
; ----- Ввод 'B'
    mov ecx,B
    mov edx,10
    call sread
; ----- Преобразование 'B' из символа в число
    mov eax,B
    call atoi ; Вызов подпрограммы перевода символа в число
    mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
    mov ecx,[A] ; 'ecx = A'
    mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
    cmp ecx,[C] ; Сравниваем 'A' и 'C'
    jg check_B ; если 'A>C', то переход на метку 'check_B',
    mov ecx,[C] ; иначе 'ecx = C'
    mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
    mov eax,max
    call atoi ; Вызов подпрограммы перевода символа в число
    mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
    mov ecx,[max]
    cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
    jg fin ; если 'max(A,C)>B', то переход на 'fin',
    mov ecx,[B] ; иначе 'ecx = B'
```

Рис. 14. Удаляем операнд



```
iokabanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07
iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:15: error: invalid combination of opcode and operands
iokabanov@10:~/work/arch-pc/lab07$
```

Рис. 15. Вывод ошибки при трансляции

```

mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.CeNdi9
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07
lab7-2.lst [-M--] 48 L:[169+41 210/227] *(13413/14768b) 0058 0x03A
168 000000E0 B801000000 <1> mov eax, 1.....
169 000000E5 CD80 <1> int 80h
170 000000E7 C3 <1> ret
2
3 00000000 D092D0B2D0B5D0B4D0- msg1 db 'Введите B: ',0h
3 00000009 B8D182D0B520423A20-
3 00000012 00 .....
4 00000013 D09DD0B0D0B8D0B1D0- msg2 db "Наибольшее число: ",0h
4 0000001C BED0B0D18CD18D0B5-
4 00000025 D0B520D187D0B8D181-
4 0000002E D0BBD0BE3A2000.....
5 00000035 32300000 A dd '20'
6 00000039 35300000 C dd '50'
7
8 00000000 <res Ah> section .bss
9 0000000A <res Ah> max resb 10
B resb 10
10
11 ..... section .text
12 global _start
13 _start:
14 ; ----- Вывод сообщения 'Введите B: '
15 mov eax ; Удален операнд
16 000000E8 E822FFFFFF error: invalid combination of opcode and operands
17 call sprint
18 ; ----- Ввод 'B'
19 mov ecx,B
20 mov edx,10
21 call sread
22 ; ----- Преобразование 'B' из символа в число
23 mov eax,B
24 call atoi ; Вызов подпрограммы перевода символа в число
25 mov [B],eax ; запись преобразованного числа в 'B'
26 00000106 A3[0A000000] ; ----- Записываем 'A' в переменную 'max'
27 mov ecx,[A] ; 'ecx = A'
28 mov [max],ecx ; 'max = A'
29 00000117 3B0D[39000000] ; ----- Сравниваем 'A' и 'C' (как символы)
30 cmp ecx,[C] ; Сравниваем 'A' и 'C'
31 jg check_B ; если 'A>C', то переход на метку 'check_B',
32 mov ecx,[C] ; иначе 'ecx = C'
33 mov [max],ecx ; 'max = C'
34 ; ----- Преобразование 'max(A,C)' из символа в число
check_B:

```

Рис. 16. Листинг с сообщением об ошибке

7.4. Задание для самостоятельной работы

1.

Задание: . Напишите программу нахождения наименьшей из 3 целочисленных переменных a , b и c . Значения переменных выбрать из табл.

7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу.

Так как в ходе лабораторной работы был получен вариант 16, создаем программу в файле task1.asm, которая находит наименьшую из целочисленных переменных $a = 44$, $b = 74$, $c = 17$. Создаем исполняемый файл и убеждаемся в корректной работе программы. (Наименьшее из чисел равно 17.)

Листинг task1.asm:

```
%include 'in_out.asm'
```

- section .data
 - msg2 db "Наименьшее число: ",0h
 - A dd 44
 - B dd 74
 - C dd 17
 -
 - section .bss
 - min resd 1 ; Резервируем место под переменную min (32 бита)
 -
 - section .text
 - global _start
 -
 - _start:
 - ; ----- Записываем 'A' в переменную 'min' как начальное значение
 - mov ecx, [A] ; ecx = 44
 - mov [min], ecx ; min = 44
 -
 - ; ----- Сравниваем текущий 'min' (A) и 'B'
 - cmp ecx, [B] ; Сравниваем 44 и 74
 - jle check_C ; Если ecx <= B, то ecx остается минимумом,
- переходим к проверке C
- mov ecx, [B] ; Иначе (если B меньше), то ecx = B
 - mov [min], ecx ; Сохраняем новый минимум
 -
 - ; ----- Сравниваем текущий 'min' и 'C'
 - check_C:
 - cmp ecx, [C] ; Сравниваем текущий минимум и 17
 - jle fin ; Если текущий минимум <= C, то идем на вывод
 - mov ecx, [C] ; Иначе (если C меньше), то ecx = C
 - mov [min], ecx ; Сохраняем C как итоговый минимум
 -

- ; ----- Вывод результата
- fin:
- mov eax, msg2
- call sprint ; Вывод сообщения 'Наименьшее число: '
-
- mov eax, [min]
- call iprintLF ; Вывод числа из переменной min
-
- call quit ; Выход

```

task1.asm      [-M--] 27 L: [ 1+40 41/ 41] *(1533/1533b) <EOF>
#include 'in_out.asm'

section .data
    msg2 db "Наименьшее число: ",0h
    A dd 44
    B dd 74
    C dd 17

section .bss
    min resd 1 ; Резервируем место под переменную min (32 бита)

section .text
    global _start

_start:
; ----- Записываем 'A' в переменную 'min' как начальное значение
    mov ecx, [A] ; ecx = 44
    mov [min], ecx ; min = 44

; ----- Сравниваем текущий 'min' (A) и 'B'
    cmp ecx, [B] ; Сравниваем 44 и 74
    jle check_C ; Если ecx <= B, то ecx остается минимумом, переходим к проверке C
    mov ecx, [B] ; Иначе (если B меньше), то ecx = B
    mov [min], ecx ; Сохраняем новый минимум

; ----- Сравниваем текущий 'min' и 'C'
check_C:
    cmp ecx, [C] ; Сравниваем текущий минимум и 17
    jle fin ; Если текущий минимум <= C, то идем на вывод
    mov ecx, [C] ; Иначе (если C меньше), то ecx = C
    mov [min], ecx ; Сохраняем C как итоговый минимум

; ----- Вывод результата
fin:
    mov eax, msg2
    call sprint ; Вывод сообщения 'Наименьшее число: '
    ....
    mov eax, [min]
    call iprintLF ; Вывод числа из переменной min
    ....
    call quit ; Выход
  
```

Рис. 17. Создаем task1.asm

```

iokabanov@10:~/work/arch-pc/lab07$ nasm -f elf task1.asm
iokabanov@10:~/work/arch-pc/lab07$ ld -m elf_i386 -o task1 task1.o
iokabanov@10:~/work/arch-pc/lab07$ ./task1
Наименьшее число: 17
iokabanov@10:~/work/arch-pc/lab07$
  
```

Рис. 18. Запуск task1

2.

Задание: Напишите программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрать из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу для значений x и a из 7.6.

В соответствии с вариантом 16 создаем программу в файле task2.asm, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x) = \begin{cases} x + 4, & x < 4 \\ ax, & x \geq 4 \end{cases}$ и выводит результат вычислений. Создаем исполняемый файл и убеждаемся в корректной работе программы на значениях x и a из таблицы 7.6 . ($x_1 = 1, a_1 = 1, f(x) = 5$; $x_1 = 7, a_1 = 1, f(x) = 7$)

Листинг task2.asm:

```
%include 'in_out.asm'
```

```
section .data
```

```
msg_x db 'Введите x: ', 0h
```

```
msg_a db 'Введите a: ', 0h
```

```
msg_res db 'Результат f(x): ', 0h
```

```
section .bss
```

```
x resd 1 ; Переменная для x
```

```
a resd 1 ; Переменная для a
```

```
res resd 1 ; Переменная для результата
```

```
buffer resb 20 ; Буфер для ввода строк
```

```
section .text
```

```
global _start
```

```

_start:

; --- Ввод x ---
mov eax, msg_x
call sprint ; Вывод "Введите x: "

mov ecx, buffer
mov edx, 20
call sread ; Ввод строки

mov eax, buffer
call atoi ; Преобразование строки в число
mov [x], eax ; Сохраняем x

; --- Ввод a ---
mov eax, msg_a
call sprint ; Вывод "Введите a: "

mov ecx, buffer
mov edx, 20
call sread ; Ввод строки

mov eax, buffer
call atoi ; Преобразование строки в число
mov [a], eax ; Сохраняем a

; --- Вычисление функции ---
; Сравниваем x и 4
mov ecx, [x]
cmp ecx, 4
jge greater_or_equal ; Если x >= 4, прыгаем на метку умножения

```

; --- Ветка: $x < 4$ (Результат = $x + 4$) ---

add ecx, 4 ; $ecx = x + 4$

mov [res], ecx ; Сохраняем результат

jmp print_res ; Пропускаем ветку умножения и идем к выводу

; --- Ветка: $x \geq 4$ (Результат = $a * x$) ---

greater_or_equal:

mov eax, [x] ; Загружаем x в eax

mov ebx, [a] ; Загружаем a в ebx

mul ebx ; $eax = eax * ebx$ ($x * a$)

mov [res], eax ; Сохраняем результат

; --- Вывод результата ---

print_res:

mov eax, msg_res

call sprint ; Вывод "Результат f(x): "

mov eax, [res]

call iprintLF ; Вывод числа

call quit ; Выход


```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.GstpaS
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07

task2.asm [-M--] 28 L:[ 1+27 28/ 68] *(773 /2122b) 1183 0x44F
#include 'in_out.asm'

section .data
    msg_x db 'Введите x: ', 0h
    msg_a db 'Введите a: ', 0h
    msg_res db 'Результат f(x): ', 0h

section .bss
    x resd 1 ; Переменная для x
    a resd 1 ; Переменная для a
    res resd 1 ; Переменная для результата
    buffer resb 20 ; Буфер для ввода строк

section .text
    global _start

_start:
    ; --- Ввод x ---
    mov eax, msg_x
    call sprint ; Вывод "Введите x: "

    ....
    mov ecx, buffer
    mov edx, 20
    call sread ; Ввод строки

    ....
    mov eax, buffer
    call atoi ; Преобразование строки в число
    mov [x], eax ; Сохраняем x

    ; --- Ввод a ---
    mov eax, msg_a
    call sprint ; Вывод "Введите a: "

    ....
    mov ecx, buffer
    mov edx, 20
    call sread ; Ввод строки

    ....
    mov eax, buffer
    call atoi ; Преобразование строки в число
    mov [a], eax ; Сохраняем a
```

Рис. 19 Создание task2.asm

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab07 — /usr/bin/mc -P /tmp/mc.pwd.GstpaS
file://10.0.2.15/home/iokabanov/work/arch-pc/lab07

task2.asm [-M--] 28 L:[ 27+41 68/ 68] *(2122/2122b) <E0F>
    call atoi ; Преобразование строки в число
    mov [x], eax ; Сохраняем x

    ; --- Ввод a ---
    mov eax, msg_a
    call sprint ; Вывод "Введите a: "

    ....
    mov ecx, buffer
    mov edx, 20
    call sread ; Ввод строки

    ....
    mov eax, buffer
    call atoi ; Преобразование строки в число
    mov [a], eax ; Сохраняем a

    ; --- Вычисление функции ---
    ; Сравниваем x и 4
    mov ecx, [x]
    cmp ecx, 4
    jge greater_or_equal ; Если x >= 4, прыгаем на метку умножения

    ; --- Ветка: x < 4 (Результат = x + 4) ---
    add ecx, 4 ; ecx = x + 4
    mov [res], ecx ; Сохраняем результат
    jmp print_res ; Пропускаем ветку умножения и идем к выводу

    ; --- Ветка: x >= 4 (Результат = a * x) ---
greater_or_equal:
    mov eax, [x] ; Загружаем x в eax
    mov ebx, [a] ; Загружаем a в ebx
    mul ebx ; eax = eax * ebx (x * a)
    mov [res], eax ; Сохраняем результат

    ; --- Вывод результата ---
print_res:
    mov eax, msg_res
    call sprint ; Вывод "Результат f(x): "

    ....
    mov eax, [res]
    call iprintf ; Вывод числа

    ....
    call quit ; Выход
```

Рис. 20. Продолжение рис. 19

```
ioakanov@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07
ioakanov@10:~/work/arch-pc/lab07$ nasm -f elf task2.asm
ioakanov@10:~/work/arch-pc/lab07$ ld -m elf_i386 -o task2 task2.o
ioakanov@10:~/work/arch-pc/lab07$ ./task2
Введите x: 1
Введите a: 1
Результат f(x): 5
ioakanov@10:~/work/arch-pc/lab07$ ./task2
Введите x: 7
Введите a: 1
Результат f(x): 7
ioakanov@10:~/work/arch-pc/lab07$
```

Рис. 21. Запуск task2

Вывод

В ходе выполнения лабораторной работы были освоены команды условного и безусловного переходов, приобретены навыки написания программ с использованием переходов. Также на практике были получены базовые знания о назначении и структуре файла листинга.